



Build for everyone

Software Engineering Manager Interview

Hello,

First of all, we are very happy you're in the interview process with us, we hope you have a good time and get excited with our people and products! Below, we will share as much as possible about how to ROCK in your interview process.

Please read carefully.



Data Structures, coding and algorithms

For the Code interview, you will be asked to review lines of code and asked for your thoughts on **style, idiomacy, efficiency, and correctness of the code in question**. **Note:** please prepare to have to work on problems of DSA and writing code as well.

Coding - You should know at least one programming language really well, preferably C++, Java, Python, Go, or C. You will be expected to know APIs, Object Oriented Design and programming, how to test your code, and come up with corner cases and edge cases for code. Note that we focus on conceptual understanding rather than memorization.

Algorithms - Approach the problem with both bottom-up and top-down algorithms. You will be expected to know the complexity of an algorithm and how you can improve/change it. Algorithms that are used to solve Google problems include sorting (plus searching and binary search), divide-and-conquer, dynamic programming/memoization, greediness, recursion or algorithms linked to a specific data structure. Know Big-O notations (e.g. run time) and be ready to discuss complex algorithms like Dijkstra and A*.

We recommend discussing or outlining the algorithm you have in mind before writing code.

Sorting - Be familiar with common sorting functions and on what kind of input data they're efficient on or not. Think about efficiency in terms of runtime and space used. For example, in exceptional cases insertion-sort or radix-sort are much better than the generic QuickSort/MergeSort/HeapSort answers.

Data structures - You should study up on as many data structures as possible. Data structures most frequently used are **arrays, linked lists, stacks, queues, hash-sets, hash-maps, hash-tables, dictionary, trees and binary trees, heaps and graphs**. You should know the data structure inside out, and what algorithms tend to go along with each data structure.

Mathematics - Some interviewers ask basic discrete math questions. This is more prevalent at Google than at other companies because counting problems, probability problems and other Discrete Math 101 situations surround us. Spend some time before the interview careers.google.com refreshing your memory on (or teaching yourself) the essentials of elementary probability theory and combinatorics. You should be familiar with n-choose-k problems and their ilk.

Graphs - Consider if a problem can be applied with graph algorithms like distance, search, connectivity, cycle-detection, etc. There are three basic ways to represent a graph in memory (objects and pointers, matrix, and adjacency list) — familiarize yourself with each representation and its pros and cons.

You should know the basic graph traversal algorithms, breadth-first search and depth-first search. Know their computational complexity, their tradeoffs and how to implement them in real code.

Recursion - Many coding problems involve thinking recursively and potentially coding a recursive solution. Use recursion to find more elegant solutions to problems that can be solved iteratively.

Practicing problems is one of the best ways to make sure you're prepared for the interview day. Our [Google Tech Dev Guide](#) offers a full library of questions similar to those you'll receive on your interview day.

- Practice Book: Cracking the Coding Interview
- Sites like to practice coding exercises like: leetcode.com, hackerrank.com,, etc
- Our [Google Tech Dev Guide](#) offers a full library of questions similar to those you'll receive on your interview day.
- Review [mock interviews](#)
- Check out our FAQs in [How We Hire](#)
- And also check [Interview Tips](#)

Extra:

- blog.google/intl/pt-br/ for overall news and updates about Google Brazil.
- <https://about.google/belonging/accessibility-resources/> for Google's accessibility and belonging initiatives.

How to Succeed

- At Google, we believe in collaboration and sharing ideas. Most importantly you'll need more information from the interviewer to analyze & answer the question to its full extent.
- It's OK to question your interviewer.
- When asked to provide a solution, first define and frame the problem as you see it.
- If you don't understand - ask for help or clarification.
- If you need to assume something - verbally check its a correct assumption!

- Describe how you want to tackle solving each part of the question.
- Always let your interviewer know what you are thinking as he/she will be as interested in your process of thought as your solution. Also, if you're stuck, they may provide hints if they know what you're doing.
- Finally, listen - don't miss a hint if your interviewer is trying to assist you!

Interviewers will be looking at the approach to questions as much as the answer:

- Does the candidate listen carefully and comprehend the question?
- Are the correct questions asked before proceeding? (important!)
- Is brute force used to solve a problem? (not good!)
- Are things assumed without first checking? (not good!)
- Are hints heard and needed?
- Is the candidate slow to grasp / solve problems? (not good!)
- Does the candidate enjoy finding multiple solutions before choosing the best one?
- Are new ideas and methods of tackling a problem sought?
- Is the candidate inventive and flexible in their solutions and open to new ideas?
- Can questioning move up to more complex problem solving?
- Talk through your thought process about the questions you are asked. In all of Google's interviews, our engineers are evaluating not only your technical abilities but also how you approach problems and how you try to solve them.
- Ask clarifying questions if you don't understand the problem or need more information. Many of the questions asked in Google interviews are deliberately underspecified because our engineers are looking to see how you engage the problem. In particular, they are looking to see which areas leap to your mind as the most important piece of the technological puzzle you've been presented with.
- Think about ways to improve the solution you'll present. In many cases, the first answer that springs to mind may not be the most elegant solution and may need some refining. It's definitely worthwhile to talk through your initial thoughts to a question and take time to compose a more efficient solution.

Google is keen to see really high quality, efficient, clear code without typing mistakes. Because all engineers (at every level) collaborate throughout the Google code base, with an efficient code review process, it's essential that every engineer works at the same high standard.

Review Code

What makes a good code review interview?

- Good self-directed problem solving skills exploring trade offs.
- Focus open ended problems towards proactive and practical solutions.
- Be confident in discussing the **underlying problems** with the code, including issues of performance, complexity, or correctness.
- Avoid pointing out individual errors line by line and instead focus on the pattern of problem(s) you're seeing in the code.
- If writing code, show that you had a **previous level of mastery** and can return to that level quickly.

- Demonstrate basic knowledge of core language libraries (even if lacking some immediate details.)
- It is a good idea to practice writing code in preparation as well as reading code, and also reviewing the Google style guides for the respective languages.

Interview: Leadership

- Project Management and Software Process
- Leadership/People Management (culture fit)
- How do you manage various scenarios that arise?
- How do you interact with partners and other teams?
- Product/Business Design Sense
- Software Engineering and Technical Achievement

Technical Leadership

We'll also assess how you lead a team technically. Our expectations are similar to Eng Leadership in the sense that you continue to dive deeper into your answers and explain the why and how. An example question could be "how do you manage technical debt?" There could also be questions around prioritization of projects and managing multiple projects at once. Again, think about your experience and how it can relate to the questions.

Management: how do you deal with common scenarios:

- Teams cooperating across geographic differences - do you have experience here and advice for yourself for future/strategies from experience/observation?
- Dealing with common performance problems amongst reports and how you'd deal with different cases (looking for individuality in the answers; not regurgitating what you've read leadership is like at Google)
- Scenarios around meeting/missing deadline

Interview: People Management: You'll receive questions directly probing your ability as a people manager that might come in the form of something like "how do you manage a low performer?" or "how do you continue to motivate a high performer". While we recognize that there may be "textbook" answers to these questions, and that's perfectly fine, we also want to know the reason behind that answer. We expect that you'll be able to go into depth and use examples of your current or past experience to dive deeper. A great way to prepare for these questions is to think about your previous leadership experience and get examples of things ready to talk about whether that be the hardest challenge you've faced, when is a time you have managed up etc.

Info on what it means to be a manager at [Google \(Project Oxygen\) here.](#)

General topics to think about:

- What have you done that demonstrated innovation?
- How did you restructure your team to handle growth, changes to organization, tracking, procedures? What would you do differently next time?

- What kind of team would you most like to have, serving customer requests, building a single product, building a suite of products, or a combination?
- Was there a time you had a big hit, something with a lot of potential impact if only you had more devs to work on it? If so, what did you do?



Interview: Organizational Strategy

If needed (your recruiter will let you know after the first Stage.

- How do you proactively create opportunities for your team / org?
- What do you think about talent across an organization? (& how to organize and plan for it)
- Long term planning: how to hire / develop skills over the next 3-5 years.
- Questions around sustaining the engineering org (vs development).
- How do you maintain technical excellence? How do you recruit top talent?
- How do you retain and grow your leadership bench?

General System Design

We expect you to lead the design discussion during this interview. You should ask clarifying and probing questions to understand what the interviewer is looking for. We know and understand that design solutions don't come together in 1 hour. Talk through your solution, make adjustments and dive deep.

General System Design Interview - Extra

- Follow a structured approach to the problem. You can define your own steps to follow while tackling such questions.
- Read/practice as many SD questions as you can. In addition to all the documents you found, like Grokking the System Design Interview on educative.io

Step	Explanation
1 - Goal	Clarifying ambiguities early in the interview is critical. Spend time clearly defining the end goals of the system to have a better chance of success. Make sure you understand the basic requirements and ask clarification questions. Start with the most basic assumptions: • <i>What is the goal of the system?</i> • <i>Who are the users of the system? Who is going to use it?</i> • <i>How are they going to use it?</i> • <i>What are the inputs and outputs of the system?</i> • <i>What do they need it for?</i>

2 - Features of the system and the APIs to interact with the system and NFRs

Describe the feature set that you'll be talking about. Try to define all the features that you think of by their importance to the user. You don't have to get it right on your first attempt, but make sure that you and your interviewer agree.

Ask clarifying questions, such as:

- *Do we want to discuss the end-to-end experience or just the API?*
- *What clients do we want to support (mobile, web, etc)?*
- *Are we focusing on the backend only or are we developing the front-end too?*
- *Will tweets contain photos and videos?*
- *Will there be any push notification for new (or important) tweets?*
- *Will users be able to search tweets?*
- *Do we need to display hot trending topics?*
- *Do we require authentication? Analytics? Integrating with existing systems?*

Decide which NFR we should focus on: consistency, availability, partitioning, scalability, latency (fast retrieval), durability, redundancy of data

Define what APIs are expected from the system. This will not only establish the exact contract expected from the system but will also ensure if we haven't gotten any requirements wrong. Some examples of APIs for our Twitter-like service will be:

- `postTweet(user_id, tweet_data, tweet_location, user_location, timestamp, ...)`
- `generateTimeline(user_id, current_time, user_location, ...)`
- `markTweetFavorite(user_id, tweet_id, timestamp, ...)`

List the NFRs (Non Functional Requirements) expected from the system. Examples:

- Consistency - Have always an accurate reply ? All users see the same data, at the same time
- Availability - System should be always capable of replying ?
- Partitioning - System continuous to function even if there is no communication between nodes ?
- Reliability - What is stored cannot be lost ?
- Low latency - Efficient response time ?
- Analytics and monitoring ?

3 - Back of the envelope - size the system (back of the envelope calculation) - and database design

It is always a good idea to estimate the scale of the system we're going to design. This will also help later when we will be focusing on scaling, partitioning, load balancing and caching.

Do some math in order to figure out the size of the system

- *What's the ratio between the reads and the writes ?*
- *How many customers in the coming year ? In 3 years time ?*
- *What scale is expected from the system (e.g., number of new tweets, number of tweet views, number of timeline generations per sec., etc.)?*
- *How much storage will we need? We will have different storage requirements if users can have photos and videos in their tweets.*
- *What network bandwidth usage are we expecting? This will be key in deciding how we will manage traffic and balance load between servers.*
- Ballpark numbers
- Back of the envelope calculations

Back-of-the-envelope analysis. This essentially means developing an intuition for the performance of different alternate designs, so that you can reject possible designs out-of-hand, or choose which alternatives to consider more carefully.

4 - High-level design

The goal of this is to outline all the important components that your architecture will need.

Sketch your main components and the connections between them. If you do this, very quickly you will be able to get feedback if you are moving in the right direction.

Start with covering the end-to-end process, based on the goals you've established.

Define the major brick that composes the system and the interfaces it has to expose !

Clarifying questions, such as:

- *Are all the user interactions there ?*
- *Have we covered all the external API ?*
- *Do we need any offline processes ?*

5 - Low-level design	This might include detailing different clients, APIs, backend services, offline processes, network architecture, data stores, and how they all come together to meet the requirements. Dig deeper into two or three major components; interviewer's feedback should always guide us to what parts of the system need further discussion. We should be able to present different approaches, their pros and cons, and explain why we will prefer one approach on the other. Remember there is no single answer; the only important thing is to consider tradeoffs between different options while keeping system constraints in mind. Questions to ask yourself ? • How should we scale the system ? ◦ Have you considered the scalability of the AppServers ? ◦ Have you considered the scaling of the Data ? ■ Since we will be storing a huge amount of data, how should we partition our data to distribute it to multiple databases? Should we try to store all the data of a user on the same database? What issue could it cause? • Have we considered the redundancy in terms of processing and data ? • Have we handled the partitioning ? How and why in this way ? • Real time or not real time ? • Do I need some caching and where ? ◦ How much and at which layer should we introduce cache to speed things up?
6 - Bottlenecks and To be discussed	Try to discuss as many bottlenecks as possible and different approaches to mitigate them. • Is there any single point of failure in our system? What are we doing to mitigate it? • Do we have enough replicas of the data so that if we lose a few servers, we can still serve our users? • Similarly, do we have enough copies of different services running such that a few failures will not trigger total system shutdown? • How are we monitoring the performance of our service? Do we get alerts whenever critical components fail or their performance degrades? List the things that come to my mind and we should talk about.
7 - Assumptions	All the constraints that we give ourselves

Coaching and prep videos:

[Leadership](#), [System Design](#), [Coding](#), [Program/Product Management](#)

A couple of important notes:

- Please prepare to interview in English, many Googlers around the world interview Brazil's candidates and vice-versa, so we cannot guarantee your interview will be in Portuguese.
- You'll need a computer with internet access and camera for your technical interview. In your confirmation email you'll receive the link to the Google Doc (a web-based word processor which lets you share your work and collaborate online) that you'll use for the coding questions in your interview.
- **Google cannot share specific feedback on any of the stages of your candidacy. We will always keep you in the loop about a positive or negative outcome, but we can not share details.**

